

MERN Stack Internship

## DAY 10 ASSIGNMENT REPORT

Name: NIKHIL MANDLOI

Course: BTech CSIT 1 year

Position : MERNStackINTERN

Allocated Date:10/06/26

Submission Date: 10/006/26

## MERN Stack Internship – Day 10 Report

### Project Title

Employee Attendance Dashboard – Component Architecture Optimization & State Management

---

### Objective

The objective of this task was to build and optimize an Employee Attendance Dashboard using React. The project focused on centralized state management using Context API, identifying and preventing unnecessary component re-renders using React.memo, understanding the lifecycle of useEffect, and explaining how React's Virtual DOM improves application performance.

---

### Technologies Used

- ☐ React.js
  - ☐ JavaScript (ES6)
  - ☐ CSS3
  - ☐ React Hooks (useState, useEffect, useContext, useMemo)
  - ☐ React Context API
  - ☐ React.memo
  - ☐ JSON Data Source
- 

### Project Overview

The Employee Attendance Dashboard displays employee information and salary statistics through reusable React components.

The application consists of:

- ☐ Dashboard Component
- ☐ EmployeeList Component
- ☐ SalaryChart Component

- ☐ EmployeeContext Provider
  - ☐ JSON Data Source
- 

## Features Implemented

### 1. Employee Data Management

Employee records were stored inside a JSON file and loaded into React state.

Sample Data:

- ☐ Rahul (IT)
  - ☐ Sneha (HR)
  - ☐ Aman (Finance)
  - ☐ Priya (Marketing)
- 

### 2. Employee List Component

The EmployeeList component displays:

- ☐ Employee Name
- ☐ Department
- ☐ Salary

The component dynamically renders employee records using the map() method.

---

### 3. Salary Summary Component

The SalaryChart component calculates and displays:

- ☐ Total Employee Salary

The calculation is optimized using useMemo() to prevent unnecessary recalculations.

---

### 4. Context API Integration

A centralized EmployeeContext was created to manage employee data.

#### Benefits:

- ☐ Eliminated prop drilling
- ☐ Centralized state management
- ☐ Improved component communication

#### Implementation:

- ☐ createContext()
  - ☐ Provider
  - ☐ useContext()
- 

### 5. React.memo Optimization

Both EmployeeList and SalaryChart components were wrapped using React.memo().

#### Benefits:

- ☐ Prevents unnecessary re-renders
  - ☐ Improves rendering efficiency
  - ☐ Enhances application performance
- 

### 6. useEffect Lifecycle Demonstration

Lifecycle stages demonstrated:

#### Mounting Phase

Executed when component loads.

Example:

- ☐ Dashboard Mounted

#### Updating Phase

Executed whenever employee state changes.

Example:

- ☐ Employees Updated

## Unmounting Phase

Cleanup function executed when component is removed.

Example:

- ☐ Dashboard Unmounted
- 

## 7. Virtual DOM Concept

React uses a Virtual DOM to improve rendering performance.

Process:

1. Create Virtual DOM copy
2. Compare old and new Virtual DOM
3. Identify changed elements
4. Update only required DOM nodes

Benefits:

- ☐ Faster rendering
  - ☐ Reduced DOM manipulation
  - ☐ Better user experience
- 

## Component Structure

src/

```
|— App.jsx
|— EmployeeList.jsx
|— SalaryChart.jsx
|— context/
|   |— EmployeeContext.jsx
|— data/
|   |— employees.json
```

└── App.css

└── main.jsx

---

## Optimization Techniques Applied

### Optimization Purpose

React.memo Prevent unnecessary renders

Context API Centralized state management

useMemo Optimize salary calculations

useEffect Lifecycle management

Virtual DOM Efficient UI updates

---

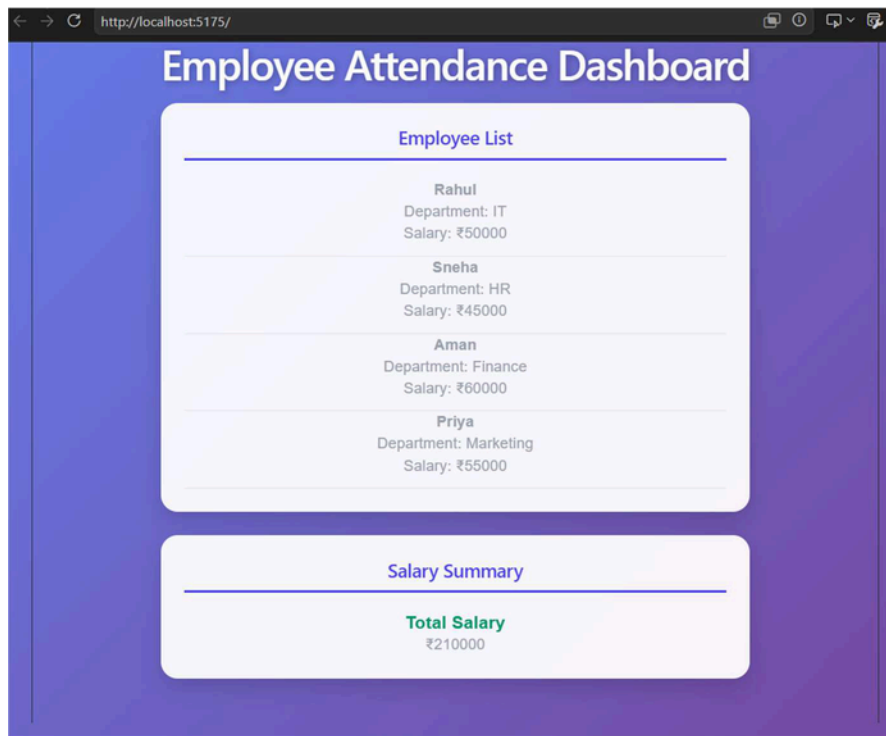
## Learning Outcomes

Through this project I learned:

- ☐ React Context API
- ☐ Centralized State Management
- ☐ Component Optimization Techniques
- ☐ React.memo Usage
- ☐ useMemo Optimization
- ☐ useEffect Lifecycle Phases
- ☐ Virtual DOM Working Mechanism
- ☐ React Rendering Behavior
- ☐ Performance Optimization Strategies

---

## Output Screenshots



```
import { createContext } from "react";

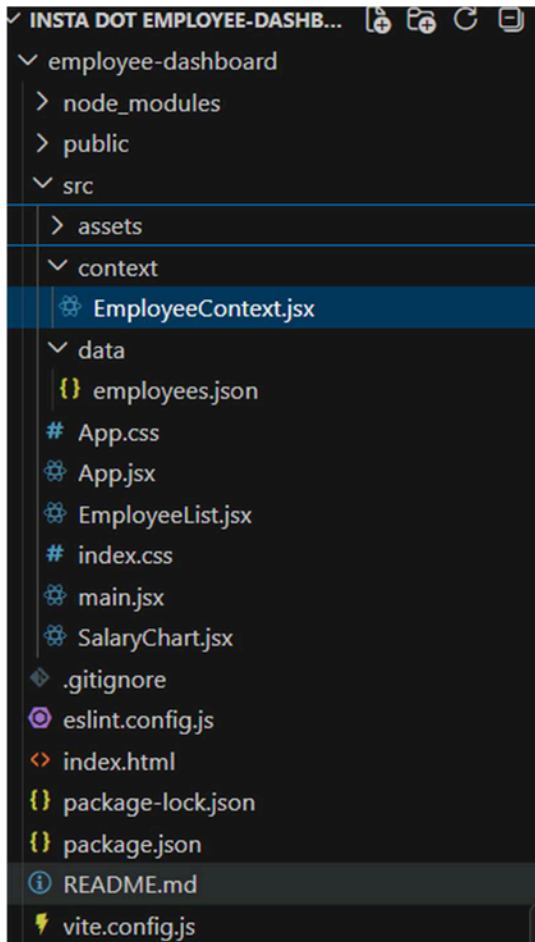
export const EmployeeContext = createContext();
```

```
employee-dashboard > src > SalaryChart.jsx > SalaryChart > memo() callback
1 C:\Users\yashm\Downloads\insta dot employee-dashboard\employee-dashboard
2 import { EmployeeContext } from "../context/EmployeeContext";
3
4 const SalaryChart = memo(() => {
5   const { employees } = useContext(EmployeeContext);
6
7   console.log("SalaryChart Rendered");
8
9   const totalSalary = useMemo(() => {
10     return employees.reduce((sum, emp) => sum + emp.salary, 0);
11   }, [employees]);
12
13   return (
14     <div className="card">
15       <h2>Salary Summary</h2>
16
17       <h3>Total Salary</h3>
18
19       <p>₹{totalSalary}</p>
20     </div>
21   );
22 });
23
24 export default SalaryChart;
```

```

employee-dashboard > src > EmployeeList.jsx > default
1  import React, { memo, useContext } from "react";
2  import { EmployeeContext } from "../context/EmployeeContext";
3
4  const EmployeeList = memo(() => {
5    const { employees } = useContext(EmployeeContext);
6
7    console.log("EmployeeList Rendered");
8
9    return (
10     <div className="card">
11       <h2>Employee List</h2>
12
13       {employees.map((emp) => (
14         <div key={emp.id}>
15           <p>
16             <strong>{emp.name}</strong>
17           </p>
18
19           <p>Department: {emp.department}</p>
20
21           <p>Salary: ₹{emp.salary}</p>
22
23           <hr />
24         </div>
25       ))}
26     </div>
27   );
28 });
29
30 export default EmployeeList;

```



Download the React DevTools for a better

EmployeeList Rendered

EmployeeList Rendered

SalaryChart Rendered

SalaryChart Rendered

Dashboard Mounted

Employees Updated

Dashboard Unmounted

Dashboard Mounted

Employees Updated

EmployeeList Rendered

EmployeeList Rendered

SalaryChart Rendered

SalaryChart Rendered

Employees Updated

> |



---

## Conclusion

The Employee Attendance Dashboard was successfully developed and optimized using React best practices. The application demonstrates centralized state management through Context API, efficient rendering using `React.memo` and `useMemo`, lifecycle handling using `useEffect`, and improved performance through React's Virtual DOM architecture. The project strengthened understanding of React component architecture, state flow, and performance optimization techniques.